

Audio Bounce

USER GUIDE

1. Introduction

1.1. [What is AudioBounce?](#)

1.2. [Quick Start](#)

1.3. [Support](#)

2. Components and Parameters

2.1. [MainController](#)

2.2. [Locator](#)

2.3. [Distributor](#)

2.4. [Audio Source Manager and Audio Register](#)

2.5. [Audio Source Volumetric](#)

2.6. [Modulation Curves](#)

2.7. [Sharable Curves](#)

2.8. [Utilities](#)

2.9. [UPDATES](#)

[Version 1.1.0](#)

1.1 What is AudioBounce?

AudioBounce is an innovative and easy-to-use realtime audio occlusion and reflection system for Unity, designed to amplify the realism of in-game sounds.

Originally born for my own in development game needs where I had to solve the problem of populating a ton of environments with realistic and immersive audio in a multitude of procedurally generated levels. Being an impossible task for a solo dev using the standard Unity triggers and volumes system I took inspiration from AAA games techniques.

AudioBounce turns the time consuming (and costly) practice of setting up audio volumes, gates, triggers for each room or area to simulate occlusion and reverberation of your game to a simple "just drag and drop the sound where you want it" and "add the AB_SoundRegister component to it" and that's it! AudioBounce will take care of everything else in real time while also adding much more needed features and realism to it automatically!

Here's a list of the main features:

Dynamic Audio Adjustments: It tweaks sound properties like occlusion and reverberation based on the 3D layout around the player.

Natural Sound Bouncing: AudioBounce positions sounds to mimic how they'd naturally echo off walls and objects.

Automatic Handling: Say goodbye to manual setups. AudioBounce detects and adapts to moving objects and changing environments in real-time. And yes, by moving objects I mean entities like enemies as well... trust me, you'll thank me later for this.

Optimized Performance: Engineered for efficiency, AudioBounce spreads its computational load across frames and threads in a highly customizable variety of ways. At a speed of 0.3ms~ with standard settings makes it possible to include this feature in games targeted for up to 240fps out of the box and can be configured to be faster and more precise at a higher cost if you want!

Beginner-Friendly: AudioBounce is ready to work with minimal setup and knowledge by design despite it's advanced capabilities. Every feature is well-documented and explained in depth with tooltips on the tool itself, welcoming both newcomers and audio pros.

In essence, AudioBounce takes care of the complex audio work at as little or as much performance

1.2 Quickstart

After installing the package the required components in the scene are essentially 3: AB_MainController, AB_ProbeDistributor (or your own version of it) and one AB_AudioSourceRegister on each AudioSource you wish AudioBounce to handle.

I've provided you two prefabs ready to go for the MainController and the ProbeDistributor in “../HSD/AudioBounce/Prefabs/”.

The AB_MainController component should be placed roughly where your character “ears” would be situated. In a first person game it would be the MainCamera, in a third person game like mine it's going to be where your character head is (you may try it on the camera as well and see if you like it better, I don't ^^), and for a 2.5D game it's at the center of the screen at the same depth of your character.

To explain why, here's a brief walkthrough of what AudioBounce does:

Every Cycle (which is how many frames long you want it to be for your game, 32 frames by default so twice a second in a 60fps game) the MainController's brain (AB_Locator) is going to take every single AudioSource with an AB_AudioSourceRegister component in range and calculate if there's a path from the sound to the player “ears” and it handles each one separately by adjusting how much occlusion that sound has, if it bounces on a surface before reaching the player and also how much reverberation needs to be applied to that sound.

Be sure to set the MainController RayCast Layer mask correctly to hit your environment and not your Player Character (probably also your enemies unless you want them to be able to block your audio, maybe you have a WALL monster... if you do that's awesome!).

The AB_ProbeDistributor is the component that's going to map in a volumetric way the scene around the player and it provides a set of points where the Audio is allowed to bounce off to reach the Player. This is one possible solution and it's highly customizable in a huge variety of ways such as shape, number of probes and complexity, to allow you to make something that suits your needs for your game and the character your player is controlling. It also comes with a prefab you can just place in your character (be sure to set the Raycast Layers correctly in here). AudioBounce also comes with a barebones Distributor class in case you wish to make your own... it's not an easy task but there might be cases where you might want to do that, the only case it comes to my mind is for a fixed camera third person game like old school Resident Evil where probably a Prebaked Probe Distributor with a list of probes that you can load per scene (or camera angle) might be preferred to save computational cost. If it were me I would just put the ProbeDistributor where the camera is for each scene and have it realtime and save me a lot of headache but you're free to do what you want! :D (let me know if you do!)

To be honest, the quickstart guide was finished before the long explanation but since you've come so far thank you for reading, now you should be having a bigger picture about AudioBounce and how it works, I hope it's going to help you make your game awesome!

1.3 Support

For any question or support you can join our Discord Server <https://discord.gg/69JxUpV2jQ>

Components and Parameters

2.1 MainController

AudioBounce's MainController is a singleton that controls the audio effects and the audio sources in the scene. It also contains the main audio adjusting logic of the tool that actually affect the sounds such as the speed at which fx can be applied to any sound, the ratio amount/distance for both occlusion and reverberation, how many audio channels you want AudioBounce to manage, the speed and amount of movement any single audio source is allowed to travel at and gizmos.

Here's the indepth explanation of all of it's parameters:

- **S Fx Blend Speed:** The speed at which the occlusion and reverb effects will blend in and out in seconds, the lower the fastest. It's set at half a second as default. You generally don't want this blending to happen too fast because of the update rate of AudioBounce being much slower than the frames per second your game is running at (by default AB updates once every 32 frames) and this amount of blending time smoothly bridges the gap between updates for a much more natural effect.
- **Modulation Curves:** The in depth description of what the curves are used for is in the ModulationCurves scriptable object. In short this allows you to change the curves used to modulate the occlusion and reverb effects in and out at a faster or slower rate depending on the amount of reverb and occlusion factor the tool calculates. The provided Curves are carefully set up to work well with the standard parameters but you might need to change it to fit your game. Be careful to test them while playing and listen well if you like the effect when you're changing it and always keep the original one as a reference and backup!
- **Audio Mixer:** This is AudioBounce's Main Audio Mixer, it will be used to apply the effects. We do provide a preconfigured one in the package so please use that one! If you really want to use your own, make sure it has the same effects and exposed parameters with the same names as the one provided. Also make sure you add each audio group of your mixer to the next parameter's list.
- **Audio Mixer Groups:** The Audio Mixer Groups that you want to use for your audio sources. The Audio Mixer we provide comes with preconfigured Audio Mixer Groups called 'Channel_nr' that you can simply add down here. If you really want to use your own, make sure it has the same effects and exposed parameters with the same names as the one provided. AudioBounce won't calculate more 32 AudioSources in your scene but you can configure it to use less if you want to.
- **Max Channels:** The maximum number of AudioSources that AudioBounce will handle. If you have more AudioSources playing at the same time, the ones that are not calculated will be ignored and play as regular Unity Audio. This is to prevent performance loss. Keep in mind that AudioBounce automatically chooses the closest audio source and will only manage those up to this max channels number. This value can be changed in runtime but it will never go above the number of Audio Mixer Channels you have set up or 32 in total.
- **Bounce Reposition Speed:** The speed at which the calculated 3D AudioSource will move to it's target position. This prevents audio souces teleportation which in game doesn't sound natural.
- **Bounce Bias Factor:** The amount of bias to apply to the bounce repositioning. 0 = no reposition (off), 1 = full bias (the audio will be positioned at bounce point). A value of 0.7 will give more bias to the bounce position but still factor in the original audio starting position which sounds more natural.
- **Disable Doppler for Bounced Audio:** If true, the doppler effect will be disabled for the instanced audio sources. This is to prevent the doppler effect from being applied when the managed Audio Source repositions. [Update 1.1.0]
- **Gizmos Data Display:** The number of AudioSources to display gizmos for.
- **Show Gizmos:** Enable or disable Gizmos for the AudioSources managed by AudioBounce.

2.2 Locator

AudioBounce's Locator is a component that calculates the Occlusion factor for the audio sources in the scene and bounces the audio based on the probes distributed by the Distributor. If MainController is AudioBounce's brain, this is it's beating heart. This is the component that's going to determine the final performance (and the cost) of your AudioBounce setup the most in conjunction with the Diffusor. Processing Mode allows you to change how AudioBounce will handle the audio sources in your scene during your update cycles. The Fixed Update method will give you fixed audio update times but flexible performance cause no matter how many AudioSource the tool has to handle it will do so in a fixed number of frames while the other one, Dynamic Update will give you dynamic audio updates but fixed performance by allowing you to specify how many frames every single AudioSource should be calculated within, making it cycle audio updates super fast when there's very little audio sources in the scene around the player but much slower when there's many.

I tend to prefer the Fixed Update method cause I've set a maximum budget of performance loss when AudioBounce is working at full capacity for my game and I can be sure the audio is going to be updated in my preferred amount of time no matter what happens. Every project has different needs though and you might have very little sounds playing at the same time so a Dynamic Update might give you extreme performance without any noticeable impact on audio update times!

Here's an indepth explanation of it's parameters:

- **Listener Offset:** This component should ideally be put on the same GameObject as the AudioListener at hears height (cases may vary depending on your needs) in order to match the listener's position and rotation. In the case where that's not possible, you can set the height offset manually.
- **Raycast Layer Mask:** The layer mask to use for the raycasts projection for 3D objects detection. This should be set to the layer that your environment is on. Best case scenario is to set this the same as your Distributor's layer mask but there might be cases where you want to use a different layer mask for the Locator.
- **Processing Mode:** Processing Mode controls how Audio Sources are processed. Fixed mode processes a set number of sources per cycle, allowing control over performance. Higher Frames Per Cycle means lower cost but more spread-out processing. Dynamic mode adapts processing based on the 'Frames Per Audio Source' setting, allowing firm control over cost achieved with flexible processing time according to the number of sources in range.
- **Fixed Frames Per:** (If Processing Mode = Fixed) Sets the number of frames to process all Audio Sources, spreading processing over multiple frames. This value remains constant regardless of the number of Audio Sources in range. It determines the balance between frame rate impact and processing time. Higher values reduce impact but increase time. Set this based on Max Audio Sources and your desired processing time frame.
- **Dynamic Frames Per Audio Source:** (If Processing Mode = Dynamic) Defines the number of frames taken to process each Audio Source. The total frames for all sources equals this value times the number of Audio Sources in range. For example, setting this to 2 with 2 Audio Sources in range will take 4 frames to process, spreading the load and reducing frame rate impact.
- **Max Distance:** The maximum distance to process Audio Sources. This is the maximum distance that Audio Sources will be processed from the listener. This is a performance optimization. Set this to the maximum distance that you want to process Audio Sources from the listener. This will reduce the number of Audio Sources processed each cycle, reducing frame rate impact. Keep in mind that far off Audio Sources are usually so faint that having Audio Bounce apply it's effects on them will probably result in a almost no audible sound so you can safely diminish the amount of computational load by ignoring them if they're too far.

2.3 Distributor

AudioBounce's ProbeDistributor is a component that distributes probes around the Character to detect 3D objects and calculate the reverb factor and occlusion factor for the audio sources in the scene. This could be entirely replaced by a custom implementation of the AB_Distributor class. You may want to do that for a Baked solution.

The standard Distributor class provided with AudioBounce is called ProbeDistributor and I've specifically made it with first and third persons games in mind but it's capable to be adapted to 2d games by changing a few parameters as well if that's what you need. It's main role is to send a significant amount of rays from the player to the geometry around it to make a virtual map of the volumes and surfaces of the environment in the scene. This is needed to determine how much reverberation the current player position might have (inside a narrow caver it's going to be higher for example) and how much occlusion an audio source is encountering before reaching the player.

Here's an indepth explanation of it's parameters:

- **Number Of Probes** : The number of probes distributed around the character. The higher the number, the more accurate the probes distribution will be. However, this will also increase the number of raycasts and the processing time. PS: This is a debug value, it will be overwritten by the number of probes generated by the probes shape settings.
- **Frames To Spread Raycasts** : The number of frames that one cycle of update takes. 1 = EveryFrame (Faster but much heavier); 12 = The load is spread across 12 frames (Slower but much more performant). Let me tell you right now, you don't need it at 1, no game needs this to be that fast. If your game runs at 60 fps, having this number set to 30 means that it will finish it's inner calculations twice per second distributing the load across 30 frames.
- **Probes Projection Radius** : This is the maximum reach of the 3D Environment scanning. Anything outside of this range will not be considered for occlusion and reverb adjustment. (It's probably going to sound very dim and quiet anyway). Keep also in mind that if this is set to reach very far you also have to increase the number of probes you would be spreading or the resolution is going to be too sparse for AudioBounce to accurately work and produce good result.
- **Hit Point Normal Offset** : This is for offsetting the probes distribution Hit Points from the surface of the objects they hit. This is useful for avoiding the probes to be placed inside the objects. It's not recommended to increase this value too much, close to the surfaces but not touching them is the right spot (the default value works perfectly in testing).
- **Layer Mask** : The layer mask to use for the raycasts projection for 3D objects detection. This should be set to the layer that your environment is on.

// Probes Shape Settings

- **Number Of Probes Per Row** : The number of points per row of the cylindrical probes distribution. The higher the number, the more accurate the probes distribution will be. However, this will also increase the number of raycasts and the processing load.
- **Number Of Rows** : The number of rows of the cylindrical probes distribution. The higher the number, the more accurate the probes distribution will be. However, this will also increase the number of raycasts and the processing load.
- **Radius** : The radius for each row of the cylindrical probes distribution.
- **Height** : The total height of the cylindrical probes distribution.
- **Offset** : The offset for the height of the cylindrical probes distribution.

2.3 Distributor

- **Height Curve** : The height curve for the cylindrical probes distribution. This is useful to modulate the row distribution in height, allowing the user to have a more dense or sparse distribution in the selected points.
- **Shrinking Factor** : Shrinking factor is useful to change the shape of the probes distribution structure by making the modulated rows smaller or bigger. This will affect the rays projection angle since the origin point is central to the component.
- **Allow Shrinking Factor Reduction** : This is useful to allow the shrinking factor to be reduced in order to avoid having shrunk rows with too many probes close to each other. Enabling this allows you to modulate the shape of the probes structure without increasing resolution by shrinking the rows. AudioBounce works better when the probes are evenly spread.
- **Minimum Shrinking Number Per Row** : This is the minimum number of probes the shrinking factor can reduce to. It limits how many probes can be removed from a single row.
- **Draw Gizmos** : These gizmos will significantly impact performance in your game but they are really useful for configuration and debugging.
- **Draw Pre Cast** : Draw the probes distribution before the raycasts are projected. Useful for editing the probes shape settings.
- **Draw Post Cast** : Draw the probes distribution after the raycasts are projected. Useful for debugging the probes distribution position.
- **Show Ray Length** : Draw a sphere with ray length of 'probesProjectionRadius'. Useful for debugging the probes distribution range.

2.4 Audio Source Manager and Audio Source Register

AudioBounce's AudioSourceManager is a singleton that keeps track of all playing audio sources in the scene. This is an automatic class and it's a required component of the MainController. This specific component acts as a satellite dish and catches all the Audio Source that gets enabled in the scene with a Audio Source Register component assigned to them. Audio Source Register is a requirement for all the Audio Sources that you wish to be handled by AudioBounce and all it does is to send a registration signal to the Audio Source Manager and register itself to the list of Audio Bounce managed sounds.

High Priority: If true, this audio source will be registered as a high priority source. This means it will always be processed by AudioBounce to keep its values updated even when it's not playing. Useful for sudden loud sounds that can be activated at any time near the player. [Update 1.1.0]

Reverb Only: If true, this audio source will only be registered for reverb calculations. It will not be processed by AudioBounce for occlusion calculations. [Update 1.1.0]

2.5 Audio Source Volumetric

AudioBounce's AudioSourceVolumetric is a component that allows you to create audio areas that the audio source can travel through. This will in fact simulate an Audio Source that occupies a volume within space in your scene rather than a single fixed point. If you've ever wondered how to make a lava river emanate the same sound across its whole surface in your "floor is lava" game, this is it. This is an extra component, it's not really needed for AudioBounce although compatible but it can solve a lot of problems you might face so I've included it ;)

2.6 Modulation Curves

This is a class that contains all the curves that smoothly modulates the audio effects in AudioBounce. Here you can find settings such as Occlusion Modulation Curve and Reverb Modulation Curve that will help you out fine tune the amount of Occlusion and Reverberation the system should apply given the returning values. AudioBounce turns geometrical calculations into fx amount and these are the curves that it uses to do it smoothly.

Here's an indepth explanation of it's parameters:

- **Occlusion Modulation Curve:** The curve used to modulate the occlusion effect. The X axis is the amount of calculated Occlusion (0 = free travel (no bouncing), 1 = completely blocked), the Y axis is the amount of occlusion actually applied to the AudioSource (0 to 1).
- **Occlusion Distance Falloff:** This curve modulates the audio source's occlusion out based on proximity and it's meant to simulate sound propagation through surfaces. It works with MIN and MAX distances to adjust the occlusion amount. A value of 0.1 at 1 means 10% less occlusion at MIN or less distance.
- **oDF_MIN:** Occlusion Distance Falloff MIN distance. The Occlusion Distance Falloff curve is at 1 at 'less than this distance' from the audio source removing about 10% of Occlusion effect on standard settings.
- **oDF_MAX:** Occlusion Distance Falloff MAX distance. The Occlusion Distance Falloff curve is at 0 at 'more than this distance' from the audio source and the full amount of Occlusion effect will be applied to the audio source.
- **Reverb Modulation Curve:** The curve used to modulate the reverb effect. The X axis is the amount of calculated Reverb (0 = open space, 1 = very small closed space), the Y axis is the amount of reverb actually applied to the AudioSource (0 to 1).
- **rDF_MIN:** Reverb Distance Falloff MIN distance. The Reverb Distance Falloff curve is at 1 at 'less than this distance' from the audio source removing about 10% of Reverb effect on standard settings. [Update 1.1.0]
- **rDF_MAX:** Reverb Distance Falloff MAX distance. The Reverb Distance Falloff curve is at 0 at 'more than this distance' from the audio source and the full amount of Reverb effect will be applied to the audio source. [Update 1.1.0]

2.7 Sharable Curves

This class is another special one. Unity curves are somewhat lacking and by having a Scriptable Object that contains an Animation Curve at least they are easy to save and share, with the added benefit that you can change it at any time and have all the component that are using it being updated with the new curve automatically.

2.8 Utilities

The Utilities class contains functions that are useful for AudioBounce operations but not strictly necessary.

ToColor32(color) : Takes a System.Drawing.Color and turns it into a Unity Color32 allowing you a much higher selection of quick colors for Gizmos (i like them pretty).

DrawSemiCircleGizmo(transform, segments, radius, color) : It's a gizmo function that semicircles at the given transform position (from two lined arrow caps to round circles depending on the number of segments). It's not used anymore but I'm leaving it there in case you might need it.

2.9 Updates

Main Controller 1.2.0

Volume Interpolation Speed: On AudioBounce 1.2.0 we've changed the way AudioBounce hooks up itself with the playing audio sources by interpolating the volume of it's own managed audio source and the original while it's moving into position (to avoid sudden pop ups and audio interruptions that can occur with non looping audio sources). This value operates as a multiplier for sFxBlendSpeed but it only affects volume increase/decrease speed.